

BekoSIRS Teslimat Sistemi Dökümantasyonu

Teslimat planlama, rota optimizasyonu, haftalık otomatik takvim ve mobil teslimat?? ak???

Bu PDF, proje kodundaki mevcut uygulamaya g?re haz?rlanm??t?r. Backend, web y?netim paneli ve mobil teslimat?? uygulamas?ndaki teslimat s?recini birlikte a??klar.

Kaynak dosya: BekoSIRS_api/reports/teslimat_sistemi_dokumantasyonu.md

Bu döküman, BekoSIRS projesindeki teslimat sisteminin nasıl çalıştığını uçtan uca açıklar. Amaç, projeyi hiç bilmeyen bir kişinin satıştan teslimata kadar olan akışı, kullanılan modelleri, endpointleri, algoritmaları, otomatik haftalık takvim davranışını ve mobil teslimatçı ekranlarının görevini anlayabilmesidir.

1. Sistem Özeti

BekoSIRS teslimat sistemi, müşteriye ürün satışı yapıldıktan sonra ürünün teslimat planına alınmasını, rota sırasının optimize edilmesini, teslimatçıya atanmasını ve teslimat sonucunun sisteme işlenmesini sağlar.

Sistem üç ana katmandan oluşur:

- ? Backend: Django ve Django REST Framework ile teslimat modelleri, API endpointleri, rota optimizasyonu ve otomatik planlama işlemleri yürütülür.
- ? Web yönetim paneli: Admin/satıcı tarafında ürün ataması, teslimat planlama, otomatik haftalık takvim, rota görüntüleme ve teslimatçı atama işlemleri yapılır.
- ? Mobil teslimatçı uygulaması: Teslimat personeli günlük rotasını görür, rotayı başlatır, navigasyon açar, teslimatı tamamlar veya sorun bildirir.

Teslimat sistemi temel olarak şu akışı izler:

```
Satış / Ürün Atama
↓
Teslimat kaydı oluşturma
↓
Günlük veya haftalık plana yerleştirme
↓
Rota optimizasyonu
↓
Teslimatçı atama
↓
Mobil uygulamada günlük rota
↓
Rota başlatma
↓
Teslim edildi / sorun bildirildi
↓
Müşteri ürün sahipliği ve bildirim
```

2. Ana Veri Modelleri

Teslimat sistemi backend tarafında `BekoSIRS_api/products/models.py` dosyasındaki modeller üzerine kuruludur.

2.1 ProductAssignment

`ProductAssignment`, müşteriye satılan veya atanan ürünü temsil eder. Teslimat süreci buradan başlar.

Önemli alanlar:

- ? `customer`: Ürünün atanacağı müşteri.
- ? `product`: Satılan veya atanmış ürün.
- ? `quantity`: Ürün adedi.
- ? `assigned_by`: Atamayı yapan admin veya satıcı.
- ? `status`: Atamanın teslimat sürecindeki durumu.
- ? `notes`: Satış veya teslimatla ilgili notlar.

Durumları:

PLANNED	Satış yapıldı, teslimat henüz planlanmadı
SCHEDULED	Teslimat tarihi belirlendi
OUT_FOR_DELIVERY	Teslimat yola çıktı
DELIVERED	Ürün teslim edildi
CANCELLED	Atama iptal edildi

Bu model satış ile teslimat arasındaki iş bağıını tutar. Teslimat tamamlanınca müşteri için ProductOwnership kaydı oluşturulur.

2.2 Delivery

Delivery, gerçek teslimat operasyonu kayıdır. Bir ProductAssignment ile bire bir ilişkilidir.

Önemli alanlar:

- ? assignment: İlgili ürün ataması.
- ? scheduled_date: Planlanan teslimat tarihi.
- ? time_window_start, time_window_end: Teslimat zaman aralığı.
- ? address: Teslimat adresi override/snapshot alanı.
- ? address_lat, address_lng: Teslimat koordinatı.
- ? status: Teslimat durumu.
- ? delivered_at: Teslim edilme zamanı.
- ? delivered_by: Teslimata atanmış veya teslimatı tamamlayan personel.
- ? depot: Başlangıç deposu.
- ? delivery_order: Rota içindeki sıra.
- ? distance_km, eta_minutes: Mesafe ve tahmini süre bilgileri.
- ? customer_phone_snapshot, address_snapshot: Teslimat anındaki müşteri iletişim/adres snapshot bilgileri.

Durumları:

WAITING	Teslimat bekliyor
OUT_FOR_DELIVERY	Teslimat yolda
DELIVERED	Teslim edildi
FAILED	Başarısız / sorunlu teslimat

2.3 DeliveryRoute

DeliveryRoute, belirli bir gün için hazırlanmış teslimat rotasıdır.

Önemli alanlar:

- ? date: Rotanın günü.
- ? assigned_driver: Rotaya atanmış teslimat personeli.
- ? status: Rota durumu.
- ? store_address, store_lat, store_lng: Depo veya mağaza başlangıç noktası.
- ? total_distance_km: Toplam rota mesafesi.
- ? total_duration_min: Toplam tahmini süre.
- ? is_optimized: Rota optimizasyonu yapıldı mı?
- ? optimized_at: Optimizasyon zamanı.

Rota durumları:

PLANNED	Rota planlandı
IN_PROGRESS	Teslimatçı rotayı başlattı
COMPLETED	Rota tamamlandı

2.4 DeliveryRouteStop

DeliveryRouteStop, bir DeliveryRoute içindeki her durağı temsil eder.

Önemli alanlar:

- ? route: Hangi rotaya ait olduğu.
- ? delivery: İlgili teslimat.

- ? `stop_order`: Rota içindeki sıra numarası.
- ? `distance_from_previous_km`: Önceki duraktan mesafe.
- ? `duration_from_previous_min`: Önceki duraktan tahmini süre.
- ? `estimated_arrival`: Tahmini varış zamanı.

Bu model sayesinde tek bir günlük rota içinde teslimatların sırası, durak mesafeleri ve süreleri saklanır.

2.5 DepotLocation

`DepotLocation`, teslimat rotalarının başlangıç noktasını temsil eder.

Önemli alanlar:

- ? `name`: Depo veya mağaza adı.
- ? `latitude, longitude`: Başlangıç koordinatı.
- ? `is_default`: Varsayılan depo mu?
- ? `created_by`: Depoyu oluşturan kullanıcı.

Bir depo varsayılan yapılırsa sistem diğer depoları varsayılan olmaktan çıkarır. Böylece aynı anda tek varsayılan depo bulunur.

3. Backend Endpoint Yapısı

Teslimat endpointleri `BekoSIRS_api/products/urls.py` içinde DRF router ile kaydedilir ve ana proje URL yapısında `/api/v1/` altına bağlanır.

Ana endpoint grupları:

```
/api/v1/assignments/  
/api/v1/deliveries/  
/api/v1/delivery-routes/  
/api/v1/delivery-person/  
/api/v1/depots/
```

3.1 Assignments Endpointleri

`ProductAssignmentViewSet`, ürün atama ve teslimat planlama işlemlerini yönetir.

Önemli işlemler:

- ? `GET /assignments/`: Ürün atamalarını listeler.
- ? `POST /assignments/`: Yeni ürün ataması oluşturur.
- ? `GET /assignments/stats/`: Atama durum istatistiklerini döndürür.
- ? `POST /assignments/{id}/schedule_delivery/`: Tek atama için teslimat tarihi belirler.
- ? `POST /assignments/batch_schedule/`: Birden fazla atamayı aynı tarihe planlar.
- ? `POST /assignments/auto_plan/`: Otomatik teslimat planı preview üretir.
- ? `POST /assignments/approve_plan/`: Preview planı onaylayıp gerçek teslimat ve rota kayıtlarını oluşturur.

3.2 Deliveries Endpointleri

`DeliveryViewSet`, teslimat kayıtlarını yönetir.

Önemli işlemler:

- ? `GET /deliveries/`: Teslimatları listeler.
- ? `GET /deliveries/?date=YYYY-MM-DD`: Tarihe göre filtreler.
- ? `GET /deliveries/?status=WAITING`: Duruma göre filtreler.
- ? `GET /deliveries/stats/`: Teslimat istatistiklerini döndürür.
- ? `GET /deliveries/by_date/?date=YYYY-MM-DD`: Belirli tarihteki teslimatları rota sırasına göre getirir.

- ? PATCH /deliveries/{id}/: Teslimat kaydını günceller.
- ? DELETE /deliveries/{id}/: Teslimat kaydını siler.
- ? POST /deliveries/assign_driver/: Seçili teslimatları teslimatçıya atar.

3.3 Delivery Routes Endpointleri

DeliveryRouteViewSet, rota oluşturma ve rota yönetimini sağlar.

Önemli işlemler:

- ? GET /delivery-routes/: Rotaları listeler.
- ? GET /delivery-routes/?date=YYYY-MM-DD: Belirli günün rotalarını getirir.
- ? POST /delivery-routes/optimize/: Seçili teslimatlar için rota optimizasyonu yapar.
- ? POST /delivery-routes/rebalance_week/: Haftalık açık teslimat planını aktif günlere göre yeniden dağıtır.
- ? DELETE /delivery-routes/{id}/: Rotayı siler, içindeki teslimatları tekrar bekleme durumuna alır.

3.4 Delivery Person Endpointleri

DeliveryPersonViewSet, mobil teslimatçı uygulaması için özel endpointleri sağlar.

Önemli işlemler:

- ? GET /delivery-person/my_route/: Giriş yapan teslimatçının bugünkü rotasını getirir.
- ? POST /delivery-person/start_route/: Bugünkü planlı rotayı başlatır.
- ? POST /delivery-person/{id}/update_status/: Teslimat durumunu günceller.

Bu endpoint grubu sadece delivery rolündeki kullanıcılar tarafından kullanılabilir.

4. Rol ve Yetki Yapısı

Teslimat sisteminde rol bazlı erişim vardır.

Admin ve satıcı:

- ? Ürün ataması oluşturabilir.
- ? Teslimat planlayabilir.
- ? Rota oluşturabilir.
- ? Rota silebilir.
- ? Teslimatçı atayabilir.
- ? Teslimat kayıtlarını yönetebilir.

Teslimat personeli:

- ? Sadece kendisine atanmış bugünkü rotayı görebilir.
- ? Rotayı başlatabilir.
- ? Kendisine atanmış teslimatların durumunu güncelleyebilir.

Müşteri:

- ? Kendi atamalarını görebilir.
- ? Teslimat durumuna göre bildirim alır.
- ? Teslim edilen ürünler ProductOwnership üzerinden müşterinin ürünlerine eklenir.

Teslimatçı yetkisi IsDeliveryPerson ile kontrol edilir. Bu permission sadece request.user.role == 'delivery' olan kullanıcıları kabul eder.

5. Manuel Teslimat Planlama

Manuel planlama iki şekilde yapılır.

5.1 Tekli Planlama

Admin veya satıcı bir ürün ataması için teslimat tarihi belirler.

Endpoint:

```
POST /api/v1/assignments/{id}/schedule_delivery/
```

Örnek body:

```
{
  "scheduled_date": "2026-05-20",
  "address": "Opsiyonel teslimat adresi"
}
```

Backend davranışı:

1. İlgili ProductAssignment alınır. 2. Aynı atama için Delivery varsa güncellenir. 3. Yoksa yeni Delivery oluşturulur. 4. Delivery.status başlangıçta WAITING olur. 5. ProductAssignment.status SCHEDULED yapılır.

5.2 Toplu Planlama

Birden fazla atama aynı teslimat tarihine alınabilir.

Endpoint:

```
POST /api/v1/assignments/batch_schedule/
```

Örnek body:

```
{
  "assignment_ids": [12, 15, 21],
  "scheduled_date": "2026-05-20"
}
```

Backend her atama için teslimat kaydı oluşturur veya mevcut teslimat tarihini günceller. Bu yöntem rota sırasını tek başına optimize etmez. Rota sırası için ayrıca /delivery-routes/optimize/ çağrısı gerekir.

6. Otomatik Teslimat Planlama

Otomatik planlama BekoSIRS_api/products/services/auto_planner.py içinde yürütülür.

Temel amaç:

- ? Planlanmamış ürün atamalarını bulmak.
- ? Müşteri adreslerini ve koordinatlarını çözmek.
- ? Teslimatları ilçelere göre mantıklı şekilde gruplamak.
- ? Aktif teslimat günlerine dağıtmak.
- ? Günlük teslimat kapasitesini dikkate almak.
- ? Her gün için rota sırasını optimize etmek.
- ? Önce preview üretmek, kullanıcı onaylarsa veritabanına yazmak.

6.1 Otomatik Planın Girdileri

/assignments/auto_plan/ endpointi şu opsiyonel parametreleri alabilir:

```
{
  "start_date": "2026-05-13",
  "allowed_weekdays": [0, 2, 4],
  "max_hours_per_day": 8,
  "depot_id": 1,
  "assignment_ids": [10, 11]
}
```

}

Parametre anlamları:

- ? `start_date`: Planlamanın başlayacağı referans tarih.
- ? `allowed_weekdays`: Teslimat yapılabilen günler. Pazartesi 0, Pazar 6.
- ? `max_hours_per_day`: Günlük maksimum çalışma süresi.
- ? `depot_id`: Başlangıç deposu.
- ? `assignment_ids`: Sadece belirli atamaları planlamak için kullanılır.

6.2 Planlanacak Atamaların Seçilmesi

Sistem şu atamaları toplar:

`status` PLANNED veya PENDING
ve henüz Delivery kaydı olmayan atamalar

Bu sayede daha önce teslimat planına alınmış ürünler tekrar planlanmaz.

6.3 Koordinat Çözümleme

Her müşteri için koordinat bulunmaya çalışılır.

Öncelik:

1. Müşterinin açık adresindeki `latitude` ve `longitude`. 2. Eğer müşteri adresinde kesin koordinat yoksa ilçenin merkez koordinatı. 3. Bunlar da yoksa atama plan dışı bırakılır ve `warnings.no_coordinates` listesine eklenir.

Bu davranış sayesinde sistem eksik koordinatlı müşterileri sessizce yanlış rotaya koymaz; kullanıcıya açık uyarı üretir.

6.4 Aynı Müşteriye Ait Ürünlerin Gruplanması

Bir müşteriye aynı dönemde birden fazla ürün satılmış olabilir. Sistem bu ürünleri ayrı ayrı fiziksel durak gibi değerlendirmez.

Örnek:

Müşteri A:
- Buzdolabı
- Çamaşır makinesi
- Televizyon

Bu üç ürün tek müşteri durağı altında gruplanır. Plan onaylanınca her ürün için ayrı Delivery kaydı oluşabilir, fakat rota seviyesinde bu teslimatlar aynı adreste ardışık durur.

Bu yaklaşım şu avantajları sağlar:

- ? Aynı adrese gereksiz ayrı rota hesabı yapılmaz.
- ? Teslimatçı aynı müşteriyi tek ziyaret gibi görür.
- ? Aynı adresteki ek teslimatların ara mesafesi 0 km olarak hesaplanabilir.

6.5 İlçe Bazlı Dağıtım

Otomatik planlama, müşterinin ilçesini `district_name` olarak alır ve planlanmamış durakları ilçeye göre gruplar.

Amaç:

- ? Aynı ilçedeki teslimatları mümkün olduğunca aynı güne koymak.
- ? Günlük rotaların coğrafi olarak dağınık olmasını azaltmak.
- ? Teslimatçının bir gün içinde gereksiz şehirlerarası geçiş yapmasını engellemek.

Basit örnek:

Lefkoşa teslimatları → Çarşamba
Girne teslimatları → Cuma
Mağusa teslimatları → Pazar

Sistem ilçeleri teslimat yoğunluğuna göre sıralar ve günlük kapasiteye sığacak şekilde günlere yerleştirir.

6.6 Günlük Kapasite Kontrolü

Kodda temel kapasite sabitleri vardır:

```
MAX_DELIVERIES_PER_DAY = 10  
MAX_HOURS_PER_DAY = 6  
AVG_SPEED_KMH = 60  
STOP_DURATION_MIN = 5
```

Günlük teslimat sayısı limiti aktiftir. Bir gün 10 teslimata ulaşırsa sistem sonraki uygun teslimat gününe geçer.

Süre limiti de hesaplanır:

toplam süre = sürüş süresi + durak servis süreleri

Sürüş süresi:

toplam mesafe / ortalama hız

Durak süresi:

durak sayısı × 5 dakika

Eğer günlük süre `max_hours_per_day` değerini aşarsa sistem planı tamamen durdurmaz; `warnings.over_time_days` içinde uyarı döndürür. Yani teslimat sayısı limiti dağıtımı doğrudan etkiler, süre limiti ise kullanıcıya risk uyarısı verir.

6.7 Aktif Teslimat Günleri

Web panelinde kullanıcı haftanın aktif teslimat günlerini seçebilir. Backend bu günleri `allowed_weekdays` olarak alır.

Örnek:

[0, 2, 4] = Pazartesi, Çarşamba, Cuma

Eğer Salı aktif değilse ve yeni teslimat Salı gününe denk gelecekse sistem onu ilk uygun aktif güne kaydırır.

6.8 Preview ve Onay Mantığı

Otomatik plan iki aşamalıdır:

1. Preview üretimi: Veritabanına yazmaz. 2. Onay: Gerçek `Delivery`, `DeliveryRoute`, `DeliveryRouteStop` kayıtlarını oluşturur.

Bu ayırım önemlidir. Kullanıcı planı görmeden sistem otomatik olarak canlı teslimat kayıtları yaratmaz.

Preview sonucu genelde şu yapıya sahiptir:

```
{  
  "days": [  
    {  
      "date": "2026-05-15",  
      "weekday": "Cuma",  
      "district_names": ["Lefkoşa"],  
      "delivery_count": 8,  
      "total_distance_km": 42.3,  
      "total_duration_min": 95,  
      "stops": []  
    }  
  ],  
  "summary": {  
    "total_deliveries": 8,  

```



```
"total_days": 1,  
"total_distance_km": 42.3,  
"depot_id": 1,  
"depot_name": "Lefkoşa Ana Depo"  
},  
"warnings": {}  
}
```

Kullanıcı onay verirse `approve_plan` çalışır.

Onay sırasında:

1. Her gün için bir `DeliveryRoute` bulunur veya oluşturulur. 2. Her ürün ataması için `Delivery` oluşturulur. 3. Müşteri adresi ve telefonu snapshot olarak teslimata yazılır. 4. Her teslimat için `DeliveryRouteStop` oluşturulur. 5. Atama durumu `SCHEDULED` yapılır. 6. Rota tekrar optimize edilip durak sırası ve toplam metrikler güncellenir.

7. Haftalık Otomatik Takvim

Web panelindeki haftalık otomatik takvim, teslimat günlerini ve açık teslimatları yönetmek için kullanılır.

Frontend tarafında bu akış `BekoSIRS_Web/src/pages/AssignmentsPage.tsx` içinde bulunur.

7.1 Yeni Satışın Otomatik Takvime Eklenmesi

Web panelinde yeni satış oluşturulduğunda şu akış çalışır:

1. Kullanıcı müşteri ve ürün seçer. 2. Frontend `/assignments/` endpointine istek atar. 3. Backend `ProductAssignment` oluşturur. 4. Frontend yeni oluşan assignment id değerini alır. 5. Sadece bu assignment için `/assignments/auto_plan/` çağrılır. 6. Backend uygun günü, ilçeyi, kapasiteyi ve rotayı hesaplar. 7. Frontend otomatik olarak `/assignments/approve_plan/` çağırır. 8. Yeni satış haftalık teslimat takvimine yerleşir.

Bu davranış `autoPlaceNewAssignment` fonksiyonu ile yapılır.

Önemli sonuç:

Yeni satış sadece `PLANNED` kalmaz;
uygun teslimat gününe otomatik yerleştirilmeye çalışılır.

7.2 Haftalık Takvimi Yeniden Dağıtma

`/delivery-routes/rebalance_week/` endpointi haftalık açık planı yeniden düzenler.

Bu işlem:

- ? Sadece açık ve bekleyen teslimatlarla ilgilenir.
- ? `WAITING` durumundaki teslimatları taşıyabilir.
- ? `OUT_FOR_DELIVERY` veya `DELIVERED` durumundaki teslimatlara dokunmaz.
- ? Pasif güne denk gelen açık teslimatları kaldırır.
- ? İlgili atamaları tekrar `PLANNED` yapar.
- ? Planlanmamış atamalarla birlikte tekrar otomatik plan oluşturur.
- ? Yeni planı onaylayıp rotaları yeniden kurar.

Bu yaklaşım, teslimat günü ayarları değiştiğinde canlı veya tamamlanmış operasyonları bozmadan açık teslimatları yeniden dağıtır.

7.3 Aktif Gün Seçiminin Etkisi

Kullanıcı haftalık ekranda örneğin Salı gününü pasif hale getirirse:

Salı günü bekleyen teslimatlar kaldırılır
ilgili atamalar tekrar `PLANNED` yapılır
otomatik plan motoru çalışır

teslimatlar ilk uygun aktif günlere dağıtılır

Bu işlem teslimat takviminin işletme günlerine göre dinamik güncellenmesini sağlar.

8. Kullanılan Rota Algoritmaları

Projede rota optimizasyonu için pratik ve hızlı algoritmalar kullanılır.

8.1 OSRM Yol Mesafesi ve Haversine Fallback

Güncel sistem önce OSRM Table API üzerinden gerçek yol ağına göre sürüş mesafesi ve sürüş süresi matrisi almaya çalışır. OSRM, OpenStreetMap verisi üzerinde çalışan açık kaynaklı bir rota motorudur.

Kullanım amacı:

- ? Depodan ilk müşteriye mesafe.
- ? İki teslimat durağı arasındaki mesafe.
- ? Toplam rota mesafesi.
- ? Teslimatlar arası tahmini sürüş süresi.

Varsayılan yapılandırma:

```
ROUTING_API_ENABLED=True
ROUTING_API_PROVIDER=osrm
ROUTING_API_BASE_URL=https://router.project-osrm.org
ROUTING_API_PROFILE=driving
ROUTING_API_TIMEOUT_SECONDS=5
```

OSRM başarılı cevap verirse mesafe ve süreler yol verisine göre hesaplanır. API erişilemezse, timeout olursa veya rota matrisi alınamazsa sistem otomatik olarak Haversine formülüne döner. Bu fallback sayesinde teslimat planlama dış servise tamamen bağımlı kalmaz.

Önemli not: <https://router.project-osrm.org> ücretsiz demo sunucudur. Geliştirme ve demo için uygundur, fakat garanti vermez ve yoğun kullanımda engellenebilir. Üretim ortamında aynı OSRM API formatını sağlayan self-hosted OSRM sunucusu veya kurumsal rota servisi kullanılması daha doğru olur.

8.2 Nearest Neighbor

Nearest Neighbor, en yakın komşu algoritmasıdır.

Mantık:

```
Depodan başla
en yakın teslimata git
oradan kalanlar içindeki en yakın teslimata git
tüm teslimatlar bitene kadar devam et
```

Avantajları:

- ? Hızlıdır.
- ? Uygulaması basittir.
- ? Küçük ve orta ölçekli teslimat listeleri için iyi başlangıç rotası üretir.

Dezavantajı:

- ? Her zaman en kısa toplam rotayı garanti etmez.
- ? İlk seçimler sonraki adımlarda rotayı uzatabilir.

8.3 2-opt

2-opt, rota optimizasyonunda kullanılan klasik bir yerel iyileştirme algoritmasıdır. Projede uydurulmuş bir isim değildir; literatürde kullanılan genel algoritma adıdır.

Mantık:

1. Mevcut rota içinden iki bağlantı seçilir. 2. Aradaki rota parçası ters çevrilir. 3. Yeni rota daha kısaysa değişiklik kabul edilir. 4. Artık iyileşme kalmayana kadar işlem tekrar edilir.

Örnek:

Depo → A → C → B → D

2-opt bunun yerine şu sırayı daha kısa bulabilir:

Depo → A → B → C → D

Projede otomatik planlama sırasında:

Nearest Neighbor ile ilk rota üretilir
2-opt ile rota iyileştirilir

Bu kombinasyon tam matematiksel optimumu garanti etmez, fakat hız ve kalite arasında iyi bir denge sağlar.

8.4 Neden Google OR-Tools Kullanılmamış?

Bu proje mevcut haliyle tek rota/gün ve sınırlı kapasite mantığına odaklanır. Nearest Neighbor + 2-opt bu kapsam için okunabilir, bakım maliyeti düşük ve yeterince hızlıdır.

Eğer ileride şu ihtiyaçlar artarsa Google OR-Tools gibi daha gelişmiş çözümler düşünülebilir:

- ? Birden fazla araç.
- ? Araç kapasitesi.
- ? Teslimat zaman aralıkları.
- ? Teslimatçı vardiyaları.
- ? Maliyet bazlı optimizasyon.
- ? Öncelikli müşteriler.

Bu durumda problem klasik Vehicle Routing Problem veya Vehicle Routing Problem with Time Windows seviyesine çıkar.

9. Rota Optimizasyonu Nasıl Kaydedilir?

Rota optimizasyonu sonucunda sistem şu kayıtları üretir:

```
DeliveryRoute
├── DeliveryRouteStop #1 → Delivery A
├── DeliveryRouteStop #2 → Delivery B
└── DeliveryRouteStop #3 → Delivery C
```

Her durak için:

- ? Sıra numarası belirlenir.
- ? Önceki duraktan mesafe hesaplanır.
- ? Önceki duraktan süre hesaplanır.
- ? Teslimatın delivery_order alanı güncellenir.

Bu sayede hem web paneli hem mobil uygulama teslimatları aynı sıra ile gösterir.

10. Teslimatçı Atama

Web panelinden rota veya teslimatlar bir teslimatçıya atanabilir.

Endpoint:

POST /api/v1/deliveries/assign_driver/

Örnek body:

```
{
  "delivery_ids": [1, 2, 3],
  "driver_id": 5
}
```

Backend davranışı:

1. driver_id ile role='delivery' olan kullanıcı aranır. 2. Seçili teslimatların delivered_by alanı bu kullanıcı yapılır. 3. Teslimatlar bir rotaya bağlıysa ilgili DeliveryRoute.assigned_driver da güncellenir.

Not: Mevcut kodda delivered_by alanı hem teslimatçı ataması hem de teslim eden kişi anlamında kullanılıyor. Pratikte mobil rota filtrelemesi bu alan üzerinden yapılıyor.

11. Mobil Teslimatçı Akışı

Mobil uygulamada teslimat personeli için ayrı route grubu vardır: BekoSIRS_Frontend/app/(delivery).

11.1 Rol Bazlı Yönlendirme

app/_layout.tsx içinde kullanıcının rolü kontrol edilir.

```
role = delivery ise → /(delivery)
diğer roller ise → /(drawer)
```

Bu sayede teslimatçı müşteri arayüzüne, müşteri de teslimatçı arayüzüne yönlendirilmez.

11.2 Günlük Rota Görüntüleme

Mobil ana ekran /delivery-person/my_route/ endpointini çağırır.

Backend sadece:

```
delivered_by = giriş yapan teslimatçı
scheduled_date = bugün
```

olan teslimatları getirir.

Dönen cevap:

```
{
  "route": {
    "id": 3,
    "total_distance_km": 42.5,
    "total_duration_min": 110,
    "status": "PLANNED",
    "stop_count": 8,
    "completed_count": 0
  },
  "deliveries": []
}
```

Mobil ekran bu bilgileri:

- ? Günün rotası.
- ? Toplam km.
- ? Tahmini süre.
- ? Tamamlanan / toplam teslimat sayısı.
- ? Teslimat listesi.

şeklinde gösterir.

11.3 Rotayı Başlatma

Teslimatçı mobil uygulamada "Rotayı Başlat" butonuna basar.

Endpoint:

```
POST /api/v1/delivery-person/start_route/
```

Backend davranışı:

1. Bugün için PLANNED durumunda ve teslimatçıya atanmış rota aranır. 2. Rota IN_PROGRESS yapılır. 3. Rotadaki tüm teslimatlar OUT_FOR_DELIVERY yapılır. 4. İlgili ürün atamaları da OUT_FOR_DELIVERY yapılır.

Bu aşamada teslimatlar artık yola çıkmış kabul edilir.

11.4 Navigasyon

Mobil harita ve detay ekranlarında teslimat koordinatları kullanılarak harita uygulaması açılır.

Desteklenen navigasyon:

- ? Google Maps.
- ? Yandex Navigasyon.

Koordinat yoksa kullanıcıya konum bilgisinin bulunamadığı söylenir.

11.5 Teslimatı Tamamlama

Teslimat detay ekranında teslimatçı teslimat kanıtı fotoğrafı çekebilir ve teslimatı tamamlayabilir.

Endpoint:

```
POST /api/v1/delivery-person/{delivery_id}/update_status/
```

Örnek body:

```
{
  "status": "DELIVERED"
}
```

Backend davranışı:

1. Teslimatın giriş yapan teslimatçıya ait olup olmadığı kontrol edilir. 2. Durum geçerliyse Delivery.status güncellenir. 3. sync_delivery_business_state çalışır. 4. Atama DELIVERED yapılır. 5. delivered_at set edilir. 6. Müşteri için ProductOwnership oluşturulur. 7. Müşteriye teslim edildi bildirimi oluşturulur.

11.6 Sorun Bildirme

Mobil detay ekranında teslimatçı şu sorunlardan birini bildirebilir:

- ? Müşteri yok.
- ? Yanlış adres.
- ? Ürün hasarlı.
- ? Diğer.

Frontend status: "ISSUE" gönderir. Backend bunu FAILED durumuna çevirir.

Sonuç:

```
Delivery.status = FAILED
ProductAssignment.status = SCHEDULED
```

Yani ürün teslim edilmemiş olur, fakat atama tamamen iptal edilmez. Daha sonra tekrar planlanabilir.

12. Müşteri Bildirimleri ve Ürün Sahipliği

Teslimat sistemi müşteri tarafına da etki eder.

12.1 Atama Bildirimi

Yeni ürün ataması oluşturulduğunda müşteriye bildirim oluşturulur.

Bildirim mantığı:

Ürün size atandı.
Teslimat planlandığında bildirim alacaksınız.

12.2 Teslim Edildi Bildirimi

Teslimat DELIVERED olduğunda müşteriye teslim edildi bildirimi oluşturulur.

Aynı anda:

ProductOwnership(customer, product)

kayıt oluşturulur. Bu kayıt müşterinin "Ürünlerim" ekranında ürünü görebilmesini sağlar.

13. Web Yönetim Paneli Özellikleri

Web tarafında teslimat yönetimi ağırlıklı olarak `BekoSIRS_Web/src/pages/AssignmentsPage.tsx` üzerinden yürür.

Öne çıkan özellikler:

- ? Yeni satış/ürün ataması oluşturma.
- ? Planlanmamış atamaları listeleme.
- ? Tekli teslimat planlama.
- ? Toplu teslimat planlama.
- ? Otomatik plan preview alma.
- ? Otomatik planı onaylama.
- ? Haftalık aktif teslimat günlerini seçme.
- ? Haftalık takvimi yeniden dağıtma.
- ? Belirli günün teslimatlarını görüntüleme.
- ? Seçili teslimatlar için rota optimizasyonu yapma.
- ? Hazırlanmış rotaları listeleme.
- ? Rota detaylarını görüntüleme.
- ? Teslimatçı atama.
- ? Hazırlanmış rotayı silme.

Rota silindiğinde teslimatlar silinmez. Rota ve durak ilişkisi kaldırılır, teslimatlar tekrar `WAITING` durumuna alınır, ilgili atamalar `SCHEDULED` yapılır.

14. Simülasyon ve Raporlama

Projede teslimat planlamasını test etmek için bir management command vardır:

`BekoSIRS_api/products/management/commands/simulate_delivery_plans.py`

Bu komut farklı senaryoları çalıştırır ve PDF raporu üretir.

Test edilen senaryolar:

- ? Aynı müşteriye birden fazla ürün.
- ? Karışık ilçeler ve aktif teslimat günleri.
- ? Pasif güne denk gelen teslimatların aktif güne kaydırılması.

- ? Mevcut rotadaki müşteriye yeni ürün eklenmesi.
- ? Koordinatı eksik müşteri.
- ? Günlük kapasite sınırı nedeniyle yoğun haftanın bölünmesi.

Mevcut PDF:

BekoSIRS_api/reports/delivery_simulation_report.pdf

15. Önemli Davranışlar ve Sınırlar

15.1 Mevcut Sistem Neleri İyi Yapıyor?

- ? Satış ve teslimat kaydını birbirine bağlı tutuyor.
- ? Aynı müşteriye ait ürünleri tek mantıksal durakta grupluyor.
- ? İlçe bazlı teslimat dağıtımını yapıyor.
- ? Günlük teslimat sayısı kapasitesini kontrol ediyor.
- ? Aktif teslimat günlerine göre haftalık plan oluşturuyor.
- ? Depo başlangıç noktasını hesaba katıyor.
- ? Rota sırasını Nearest Neighbor + 2-opt ile optimize ediyor.
- ? Mobil teslimatçı uygulamasıyla saha akışını destekliyor.
- ? Teslimat tamamlanınca müşteri ürün sahipliğini oluşturuyor.
- ? Bildirim ve audit log mekanizmalarıyla olayları takip ediyor.

15.2 Mevcut Sınırlar

- ? Varsayılan OSRM entegrasyonu gerçek yol mesafesi/süresi döndürür; ancak demo sunucu kullanılıyorsa servis garantisi yoktur.
- ? OSRM trafik verisi kullanmaz; rota yol ağına göre hesaplanır, canlı trafik veya yol kapalı bilgisi içermez.
- ? Günlük süre aşımı için uyarı üretir, fakat süre aşımını her durumda otomatik parçalamaz.
- ? Tek rota/gün mantığı baskındır; çok araçlı tam VRP çözümü yoktur.
- ? Teslimat kanıtı fotoğrafı mobilde alınır, fakat mevcut endpoint fotoğraf dosyasını backend'e yükleyen ayrı bir payload işlemi yapmaz.
- ? delivered_by alanı hem atanan teslimatçı hem teslim eden kişi gibi kullanılır; ileride assigned_driver ve delivered_by ayrımı Delivery seviyesinde netleştirilebilir.
- ? Teslimatçı bir teslimatı tamamladığında mevcut kodda rota tamamlama kontrolü erken return nedeniyle çalışmayabilir. Bu nedenle tüm teslimatlar tamamlandığında rota durumunun otomatik COMPLETED yapılması ayrıca düzeltilmelidir.

16. Örnek Uçtan Uca Senaryo

Senaryo

Satıcı, Ahmet isimli müşteriye bir buzdolabı satıyor.

Adım 1: Satış Oluşturulur

Web panelinden müşteri ve ürün seçilir. Backend ProductAssignment oluşturur.

Durum:

ProductAssignment.status = PLANNED

Adım 2: Otomatik Plan Çalışır

Frontend yeni assignment id ile otomatik plan çağırır.

Sistem:

- ? Ahmet'in adres koordinatını bulur.
- ? İlçesini belirler.
- ? Aktif teslimat günlerini kontrol eder.
- ? Günlük kapasiteyi kontrol eder.
- ? Uygun güne yerleştirir.
- ? Rota sırasını optimize eder.

Adım 3: Plan Onaylanır

Frontend planı onaylar.

Veritabanında:

```
Delivery oluşturulur
DeliveryRoute oluşturulur veya mevcut rotaya eklenir
DeliveryRouteStop oluşturulur
ProductAssignment.status = SCHEDULED
Delivery.status = WAITING
```

Adım 4: Teslimatçı Atılır

Admin/satıcı rotaya teslimatçı atar.

```
Delivery.delivered_by = teslimatçı
DeliveryRoute.assigned_driver = teslimatçı
```

Adım 5: Teslimatçı Mobilde Rotayı Görür

Teslimatçı uygulamaya girer.

/delivery-person/my_route/ endpointi bugünkü teslimatları döndürür.

Adım 6: Rota Başlatılır

Teslimatçı "Rotayı Başlat" butonuna basar.

```
DeliveryRoute.status = IN_PROGRESS
Delivery.status = OUT_FOR_DELIVERY
ProductAssignment.status = OUT_FOR_DELIVERY
```

Adım 7: Teslimat Tamamlanır

Teslimatçı müşteri adresine gider, teslimat kanıtı alır ve "Teslim Edildi" yapar.

```
Delivery.status = DELIVERED
ProductAssignment.status = DELIVERED
ProductOwnership oluşturulur
Müşteriye bildirim oluşturulur
```

Sonuç olarak müşteri ürünü artık kendi ürünleri arasında görür.

17. Geliştirme İçin Öneriler

Sistem mevcut haliyle küçük ve orta ölçekli teslimat planlama için anlaşılır ve uygulanabilir bir yapıdadır. Daha ileri seviye için şu geliştirmeler düşünülebilir:

- ? Delivery.assigned_driver alanı eklenip delivered_by yalnızca teslimatı tamamlayan kişi olarak kullanılabilir.

- ? Teslimat kanıtı fotoğrafı backend'e dosya olarak yüklenebilir.
- ? Tüm teslimatlar tamamlandığında rota durumunu COMPLETED yapan blok erken return sonrası çalışacak şekilde düzenlenebilir.
- ? Süre aşımı oluşan günler otomatik olarak bölünebilir.
- ? Üretim kullanımı için public demo yerine self-hosted OSRM veya SLA sunan rota API'si kullanılabilir.
- ? Çok araçlı ve zaman pencereli dağıtım için Google OR-Tools değerlendirilebilir.
- ? Teslimat başarısız olduğunda yeniden planlama akışı ayrı bir ekran olarak güçlendirilebilir.

18. Kısa Sonuç

BekoSIRS teslimat sistemi satıştan teslimata kadar bütün süreci kapsayan pratik bir mimariye sahiptir. Mevcut yapı; ürün atama, teslimat kaydı, otomatik haftalık planlama, ilçe bazlı dağıtım, günlük kapasite kontrolü, rota optimizasyonu, teslimatçı atama, mobil teslimatçı akışı ve teslimat sonrası müşteri ürün sahipliği adımlarını birbirine bağlar.

Kullanılan algoritmik yaklaşım:

- Aynı müşteri gruplama
- + İlçe bazlı dağıtım
- + Günlük kapasite kontrolü
- + OSRM yol mesafesi/süresi
- + Haversine fallback
- + Nearest Neighbor
- + 2-opt rota iyileştirme

Bu yaklaşım, projeye yeni ağır bağımlılıklar eklemekten teslimat operasyonu için hızlı, okunabilir ve sürdürülebilir bir çözüm sağlar.